# Nesne Temelli Programlama (OOP) Prensipleri

Bu rapor, Nesne Temelli Programlama (OOP) yaklaşımının temel yapıtaşlarını, ezbere dayalı tanımlardan ziyade "problem çözme" odaklı bir bakış açısıyla incelemektedir. Kavramların günlük hayattaki karşılıkları ve yazılım projelerindeki kullanım amaçları küçük kod örnekleriyle desteklenmiştir.

## 1. Sınıf (Class) ve Nesne (Object)

> **Neden Kullanılır?** Gerçek dünyadaki karmaşık yapıları dijital ortama aktarırken, aynı özelliklere sahip yapıları tek tek yazmak yerine bir şablon oluşturarak kod tekrarını önlemek için kullanılır.

**Günlük Hayat Örneği:** Bir evin mimari planı (Sınıf), o plana bakılarak inşa edilmiş fiziksel ev ise nesnedir (Object). Aynı plandan yüzlerce ev (nesne) üretilebilir.

```
// Sınıf (Şablon)
class Araba {
    public string Renk;
    public void Calistir() { /* Motoru başlat */ }
}
// Nesne Oluşturma (Somutlaştırma)
Araba benimArabam = new Araba();
benimArabam.Renk = "Kırmızı";
```

## 2. Constructor (Yapıcı Metot) ve Nesne Oluşturma Mantığı

> **Hangi Problemi Çözer?** Bir nesne hafızada (RAM) yaratıldığı anda, başlangıç ayarlarının (ilk değerlerin) otomatik ve güvenli bir şekilde yapılmasını sağlar. "Eksik verili" nesnelerin sisteme girmesini engeller.

**Günlük Hayat Örneği:** Yeni bir banka hesabı açarken, bankanın sizden isim ve TC kimlik no gibi bilgileri *en başta* zorunlu olarak istemesidir. Bu bilgiler olmadan hesap (nesne) açılamaz.

```
class Kullanici {
    public string KullaniciAdi;

    // Constructor (Sınıf adıyla aynı)
    public Kullanici(string ad) {
        KullaniciAdi = ad; // Nesne doğarken değer atanır
    }
}
Kullanici k = new Kullanici("Ahmet");
```

## 3. Access Modifiers (Erişim Belirleyiciler)

Nesnelerin içindeki verilere kimlerin ulaşabileceğini kontrol eder. Veri güvenliğini sağlamak (yanlışlıkla değiştirilmeyi önlemek) için kullanılır.

- **public:** Herkese açıktır. (Örn: Bir kafenin müşteri giriş kapısı)
- **private:** Sadece tanımlandığı sınıf içinden erişilebilir. (Örn: Kafenin sadece çalışanlara özel mutfağı)
- **protected:** Sınıfın kendisi ve ondan miras alan sınıflar erişebilir. (Örn: Aile şirketinin kasasına sadece aile üyelerinin erişebilmesi)

## 4. Encapsulation (Kapsülleme)

> **Neden Kullanılır?** Sınıfın iç işleyişini ve hassas verilerini dış dünyadan gizlemek, verilere sadece izin verilen kurallar çerçevesinde (Getter/Setter ile) erişilmesini sağlamak için kullanılır. Kontrolsüz veri değişimini engeller.

**Günlük Hayat Örneği:** Bir kahve makinesi. Siz sadece düğmeye basarsınız (public metot), içerideki suyun nasıl ısındığını veya çekirdeğin nasıl öğütüldüğünü (private veriler/işlemler) görmezsiniz ve müdahale edemezsiniz.

```
class BankaHesabi {
    private decimal bakiye; // Dışarıdan doğrudan erişilemez!

    // Bakiye sadece bu kurallı metot ile artırılabilir
    public void ParaYatir(decimal miktar) {
        if (miktar > 0) {
            bakiye += miktar;
        }
    }
}
```

## 5. Abstraction (Soyutlama)

> **Hangi Problemi Çözer?** Karmaşıklığı yönetmek için detayları gizler ve sadece gerekli olan özellikleri ön plana çıkarır. Bir sistemin "nasıl" çalıştığına değil, "ne" yaptığına odaklanmamızı sağlar.

**Günlük Hayat Örneği:** Arabayı kullanırken direksiyon ve pedalları kullanırız. Motorun içindeki yanma odasında benzinin nasıl patladığını bilmek zorunda değilizdir. Detaylar bizden soyutlanmıştır.

## 6. Interface ve Abstract Class Kavramları

İkisi de soyutlama sağlar ancak kullanım amaçları farklıdır:

- **Abstract Class (Soyut Sınıf):** Ortak özellikleri olan nesneler için bir "temel (base)" oluşturur. İçinde hem bitmiş kodlar hem de sadece şablonu çizilmiş boş metotlar barındırabilir. (Örn: `Hayvan` sınıfı. Uyumak metodu her hayvanda aynıdır, yazılabilir. Ancak SesCikar metodu boştur, alt sınıfa bırakılır.)
- **Interface (Arayüz):** Bir sınıfın "hangi yeteneklere" sahip olması gerektiğini belirten bir "sözleşmedir". (Örn: `IUcabilir` arayüzü. Hem Kuş hem de Uçak bu arayüzü uygulayabilir ama aralarında bir akrabalık yoktur.)

## 7. Inheritance (Kalıtım / Miras)

> **Neden Kullanılır?** "IS-A (Bir ...dır)" ilişkisi kurarak ortak özellikleri üst bir sınıfta toplayıp kod tekrarını önler. Yeni özellikleri mevcut kodları bozmadan sisteme eklemeyi sağlar.

**Günlük Hayat Örneği:** Kedi ve Köpek birer Hayvandır. Nefes alma ve yemek yeme özelliklerini "Hayvan" üst sınıfından miras alırlar, böylece bu kodları her iki hayvana da tekrar yazmayız.

```
class Personel {
    public string Ad;
    public void MaasAl() { /*...*/ }
}
// Yazilimci, Personel'in tüm özelliklerini miras alır
class Yazilimci : Personel {
    public string BildigiDiller;
}
```

## 8. Polymorphism (Çok Biçimlilik)

> **Hangi Problemi Çözer?** Aynı isimdeki bir işlemin, farklı nesnelerde farklı şekillerde çalışmasını sağlar. Yeni bir tür eklendiğinde if-else bloklarını uzatmak yerine, sistemin otomatik olarak doğru metodu çağırmasını sağlayarak kodu esnekleştirir.

**Günlük Hayat Örneği:** Bir orkestra şefi orkestraya "Çal" komutu verir. Keman yayı titreterek, davul vurarak, flüt ise üflenerek çalar. Komut aynıdır, ancak her enstrümanın uygulama biçimi farklıdır.

```
class Hayvan {
    public virtual void SesCikar() { Console.WriteLine("Ses yok"); }
}
class Kedi : Hayvan {
    public override void SesCikar() { Console.WriteLine("Miyav"); }
}
class Kopek : Hayvan {
    public override void SesCikar() { Console.WriteLine("Hav Hav"); }
}
```

## 9. Composition ve Inheritance Arasındaki Fark

Yazılım dünyasında "Kalıtım (Inheritance) yerine Kompozisyonu (Composition) tercih et" kuralı yaygındır.

- **Inheritance (Kalıtım - IS-A):** Sıkı bir bağdır. "Otomobil bir Araçtır". Eğer Araç sınıfında hata yaparsanız tüm otomobiller bozulur. Hiyerarşi büyüdükçe yönetilmesi zorlaşır.
- **Composition (Kompozisyon - HAS-A):** Parçaları birleştirme mantığıdır. "Otomobil bir Motora **sahiptir**". Nesneler birbirinin içine yerleştirilir. Daha esnek ve güvenli bir tasarımdır.

```
// Inheritance (Kalıtım) Yaklaşımı
class UcanAraba : Araba { /* Araba'nın her şeyini alır, uçmayı ekler */ }

// Composition (Kompozisyon) Yaklaşımı (Daha Esnek)
class Araba {
    private Motor arabaMotoru; // Araba motor sınıfını KULLANIR
    private Pervane arabaPervanesi;
}
```

## 10. Nesneler Arasındaki İlişkiler

Gerçek bir projede nesneler tek başlarına yaşamazlar, birbirleriyle iletişim halindedirler:

- **Association (Birliktelik):** İki bağımsız nesnenin birbirini tanımasıdır. (Örn: Öğretmen ve Öğrenci. Öğretmen okuldan ayrılsa da öğrenci var olmaya devam eder.)
- **Aggregation (Toplama):** "Bütün-Parça" ilişkisidir ama parça tek başına yaşayabilir. (Örn: Üniversite ve Bölümler. Üniversite kapansa da Bölümler teorik olarak baka bir kuruma bağlanabilir.)
- **Composition (Oluşturma):** Sıkı bir "Bütün-Parça" ilişkisidir. Bütün yok olursa parça da yok olur. (Örn: Ev ve Odalar. Evi yıkarsanız odalar da yok olur.)
- **Dependency (Bağımlılık):** Bir nesnenin, metodunun içinde geçici olarak başka bir nesneyi kullanmasıdır (Örn: RaporOlusturucu sınıfının Yazici nesnesini kullanması).